

Glosario técnico – JavaScript moderno (ES6+)

Variables y ámbito

let

Declarador de variables con **ámbito de bloque (block scope)**. Evita problemas de hoisting asociados a `var` y está sujeto a la **zona muerta temporal (Temporal Dead Zone, TDZ)**.

const

Declarador de constantes con ámbito de bloque. Impide la **reasignación de la referencia**, pero no la mutación de estructuras complejas (objetos y arrays).

Scope (ámbito)

Contexto de visibilidad de una variable. En JavaScript moderno se distinguen:

- Scope global
- Scope de función
- Scope de bloque

Temporal Dead Zone (TDZ)

Intervalo en el que una variable declarada con `let` o `const` existe pero no puede ser accedida antes de su inicialización.

Hoisting

Mecanismo mediante el cual el motor de JavaScript eleva declaraciones al inicio del contexto de ejecución. En ES6, el comportamiento es más restrictivo debido a TDZ.

Funciones

Arrow Function (=>)

Forma abreviada de declarar funciones. Carece de:

- `this` propio (usa binding léxico)
- `arguments`
- `new` (no es constructora)

Binding léxico de `this`

Comportamiento por el cual una función hereda el valor de `this` de su contexto envolvente, en lugar de definirlo dinámicamente.

Función de primera clase (First-class function)

Función tratada como un valor: puede asignarse a variables, pasarse como argumento o devolverse desde otra función.

Función pura

Función determinista que:

- No tiene efectos secundarios
- Retorna el mismo resultado para la misma entrada

Estructuras y sintaxis moderna

Destructuring (desestructuración)

Mecanismo que permite extraer valores de objetos o arrays en variables independientes de forma declarativa.

```
const { nombre } = usuario;
```

Template Strings (Template Literals)

Literales de cadena delimitados por backticks (`) que permiten:

- Interpolación (`\${}`)
- Multilínea
- Expresiones embebidas

Spread Operator (...)

Operador que expande elementos de:

- Arrays
- Objetos iterables

Permite clonación superficial y composición de estructuras.

Rest Operator (...)

Recoge un número indefinido de parámetros o elementos en una estructura agrupada.

Modularidad

Módulo ES (ES Module)

Unidad de código encapsulado que exporta funcionalidad reusable mediante `export` e importa dependencias mediante `import`.

Export nombrado (Named export)

Exportación explícita de múltiples elementos de un módulo.

```
export const PI = 3.14;
```

Export por defecto (Default export)

Exportación principal de un módulo, permitiendo un único valor por archivo.

Encapsulación

Principio de diseño que restringe el acceso directo al estado interno de un módulo o componente.

Closures y contexto

Closure (clausura)

Estructura que resulta cuando una función accede a variables de su entorno léxico incluso después de que dicho entorno haya finalizado su ejecución.

Entorno léxico (Lexical Environment)

Estructura interna del motor que contiene variables accesibles en el contexto actual.

Contexto de ejecución (Execution Context)

Unidad de evaluación en JavaScript que incluye:

- Variable Environment
- Scope Chain
- `this`

Programación funcional

Inmutabilidad

Principio por el cual los datos no se modifican, sino que se crean nuevas versiones.

Función de orden superior (Higher-order function)

Función que:

- Recibe otras funciones como argumento
- Devuelve una función

Composición de funciones

Técnica que combina múltiples funciones para producir una nueva función más compleja.

Declaratividad

Estilo de programación que describe *qué* se quiere hacer en lugar de *cómo* hacerlo.

Arrays avanzados

Array iterable

Estructura que puede ser recorrida mediante iteradores (`for...of`, `map`, etc.).

`map()`

Método que transforma cada elemento de un array y devuelve un nuevo array.

`filter()`

Método que selecciona elementos según una condición y devuelve un nuevo array.

`reduce()`

Método que acumula valores del array en un único resultado mediante una función reductora.

Mutabilidad vs Inmutabilidad en arrays

- Mutables: `push`, `splice`
- Inmutables: `map`, `filter`, `slice`

Objetos avanzados

Objeto literal mejorado (Enhanced Object Literal)

Sintaxis simplificada en ES6 para declarar objetos:

- Propiedades abreviadas
- Métodos sin palabra `function`
- Claves dinámicas

Propiedad computada (Computed Property Name)

Permite definir claves dinámicamente en objetos.

```
const obj = { [clave]: valor };
```

Clonación superficial (Shallow copy)

Copia de un objeto donde las referencias internas se mantienen.

Referencia vs valor

- Tipos primitivos → copia por valor
- Objetos → copia por referencia

Conceptos transversales

Motor JavaScript (JS Engine)

Sistema que interpreta y ejecuta código JS (ej. V8). Incluye parser, compilador JIT y runtime.

Call Stack

Estructura LIFO donde se gestionan las llamadas a funciones.

Event Loop

Mecanismo que coordina la ejecución de tareas síncronas y asíncronas.

Asincronía (concepto base, preludio)

Capacidad de ejecutar operaciones no bloqueantes fuera del flujo principal.

Uso profesional en contexto Ionic

En aplicaciones reales con Ionic:

- Los **módulos ES** estructuran componentes
- Los **closures** aparecen en servicios y gestión de estado
- Las **arrow functions** son habituales en callbacks
- La **programación funcional** se usa en manipulación de datos (APIs)
- La **inmutabilidad** es clave para evitar efectos secundarios en UI